

@property: 定義一個與私有變數__a與同名的物件方法a()。差別在呼叫不用加括號。
也可以不定義為同名，同樣呼叫不用加括號

建好後，才可新增@a.setter、@a.deleter

```
class GetSet(object):
    def __init__(self, value):
        self._attrval = value
    @property
        # getting方法
    def var(self):
        # print(cc.var)，是呼叫此方法印出
        print('Getting the "var" attribute')
        return self._attrval

    @var.setter
        # 允許cc.var = 10，後續print(cc.var)指向上面
    def var(self, value):
        print('setting the "var" attribute')
        self._attrval = value

    @var.deleter
    def var(self):
        print('deleting the "var" attribute')
        self._attrval = None
```

繼承: 令開發快速，重新使用(reuse)。往下、衍生傳承

B -> A <- C；B、C繼承A；class B(A)，class C(A)

如果A為5，則B、C會>=5，例如B為7(5+2)，C為9(5+4)

抽象: 將共同東西往上(形而上)抽出來成概念，用於分門別類、歸類

先有抽象後有繼承(動作)，雜物 -> 抽象 -> 得到概念 -> 繼承給新事物

零件化、模組化就是為了reuse也就是繼承

單一繼承 -> 父子類別，多重繼承 -> 基礎(超)衍生(次)類別

只有資料欄位和非專用方法才能被子類別存取

專用方法: 私有，僅能在父類別存取

繼承模組裡的class: class New(module.class)

衍生類別有自己和基礎的所有性質和方法

class A(object) -> object: 所以類別的基礎類別，亞當夏娃，可省略；

class A = class A() = class A(object)

呼叫同名屬性順序: 物件屬性 -> 類別屬性 -> 基礎類別屬性，

跟區域~全域變數往上往左搜尋一樣，都是一層層往外

str(foo)->Polygon->__str__->str(pt)，pt是point物件，str會先搜索該物件是否有自訂__str__方法

Python的內建類型(類別)都有內建(魔法)__str__方法輸出正確字串

__init__: 建構函式不用return也會自動回傳值

__str__: 在str()、print()、fstring等上面才會被調用對應類別的__str__方法，不會在物件建立時自動執行。若類別未定義__str__方法，會使用預設來自object類別的__str__方法，返回物件的類別和記憶體位址。

多重繼承:解決方法Method Resolution Order(MRO)

A <- C -> B

多層繼承: 巢狀繼承

最基本的類別: object，裡面內建很多方法

可用issubclass(list, object)、isinstance(5.5, object)判斷關係

先深而後退，有共同繼承最後走，由左至右

共同繼承: 最後搜尋

.__mro__屬性或者.mro()方法，查看呼叫順序

A(X,Y)、B(Y,Z): M -> B -> A -> X -> Y -> Z，X移除後Y為兩項榜首所以先於Z

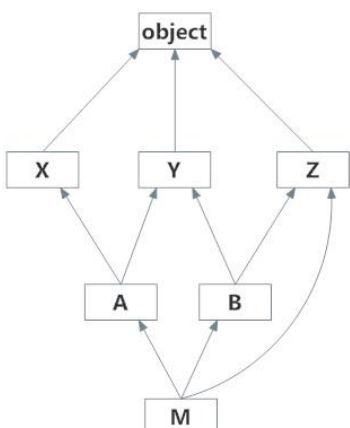
A(X)、B(Y,Z)：M -> B -> Y -> A -> Z -> X

A(Y,X)、B(Y,Z): M -> B -> A -> Y -> Z -> X

A(X,Y)、B(Z,Y): M -> B -> A -> Z -> X -> Y

3. B Y Z、A Y X、Z -> M B；Y Z、A Y X、Z -> 有一個Y排在第二個所以 M B A；

Y Z、Y X、Z -> Y都排在第一個所以 M B A Y；Z、X、Z -> M B A Y Z X



衍生類別的起始設定方法通常執行直屬基礎類別的起始設定方法

super().method(args): 由父類別開始找尋方法執行

self.method(args): 呼叫物件方法

class.method(self, args): 呼叫類別方法

覆寫: 並非改寫，呼叫順序不同。子類別設置父類別同名方法，且優先返回

運算子覆寫: 建立新的運算子魔法方法，如自訂__add__覆蓋原本的內建add()

例外處理: 自訂例外1. raise class, instance。2. raise instance

終端進入虛擬環境:

先conda activate base

後在base環境下進入conda activate Python_Code_P311

終端自動進入: 根據當下Python 解譯器終端會進入對應環境

在專案下設置解釋器會自動儲存(沿用)設定

在.vscode/settings.json 中添加以下設置:

```
{"python.pythonPath": "C:\\Users\\User\\.conda\\envs\\Python_Code_P311\\python.exe",
"python.terminal.activateEnvironment": true,
}
```

VS code需在專案下建立.vscode/settings.json

以下皆為終端操作:

- 1.設置mysite專案django-admin startproject mysite
- 2.cd mysite
- 3.進入manage.py那一層的目錄
- 4.python manage.py runserver # ctrl點擊網址，看到火箭伺服器啟動成功